

EDA Agent — Technical Report

Motivation

Existing EDA workflows are typically designed around specific datasets, limiting their generalisability and practical utility. A dataset-agnostic EDA agent addresses this by decoupling the analytical logic from the data structure, enabling automated exploratory analysis across arbitrary tabular datasets. This generalisation is critical for real-world deployment, where users cannot be expected to modify or retrain the system for each new dataset.

Problem Statement

This system is an LLM-powered agentic framework that performs automated Exploratory Data Analysis on any tabular dataset uploaded by the user. Rather than requiring manual analysis or dataset-specific scripts, the agent autonomously selects and executes the appropriate analytical tools covering data quality, statistical summaries, visualisations, and preprocessing recommendations and returns a structured, actionable report. This system is designed for data scientists, researchers, and analysts who need rapid, reliable insights from unfamiliar datasets without writing custom EDA code for each one.

System Design

- The UI exposes the agent as a conversational interface, allowing users to ask specific questions rather than receiving a fixed automated report.
- The agent is built on a tool-based architecture comprising 17 specialised analytical functions, each implemented as a LangChain tool using the closure pattern. This allows the dataset to be injected at runtime without modifying individual tool definitions, maintaining clean separation between data and logic.
- API key management was moved from server-side environment secrets to user-provided runtime input, enabling public deployment without centralised credential storage. This introduces a stateless initialisation pattern where the LLM client and agent are recreated per session

- Users can select from multiple Gemini model tiers at runtime, enabling cost-aware deployment across free and paid API quotas.

Implementation Details

- A raw ReAct loop was first implemented without any frameworks as a baseline, before migrating to LangChain to validate the architectural approach.
- Dataset ingestion is handled via the Hugging Face datasets library, which provides a unified API for loading any publicly available tabular dataset by name. The resulting dataset is converted to a pandas DataFrame to maintain compatibility with the existing tool suite.
- Dataset ingestion includes a normalisation step that converts common placeholder strings such as '?', 'NA', and 'unknown' to NaN at the point of loading. This ensures consistent missing value detection across all tools regardless of how the source dataset encodes missing data.
- Memory persistence was implemented via LangChain's MemorySaver, enabling the agent to maintain conversation state across multiple user queries within a session. This allows iterative, context-aware analysis rather than treating each question independently.
- The system was deployed to Hugging Face Spaces using Gradio 6.0. Environment differences between Kaggle and Hugging Face required adapting plot rendering from inline notebook display to server-side file saving and Gradio image components.

Results

DataBuddy was successfully deployed as a conversational web interface on Hugging Face Spaces, allowing users to load any Hugging Face dataset by name and ask arbitrary natural language questions about it. Visualisation tools including histograms, boxplots, correlation heatmaps, and group target plots are rendered directly in the chat interface alongside text analysis.

The LangChain implementation successfully replicated the raw ReAct baseline with significantly less code. Memory persistence was validated through multi-turn continuity testing, confirming that the agent maintains context across multiple queries within a session. Streaming output confirms correct tool selection and reasoning

transparency at each agent step.

Generalisation was validated across three datasets of varying complexity — Iris, Titanic, and Adult Census Income — without any modifications to the codebase. The agent correctly identified dataset-specific characteristics, selected appropriate tools, and produced accurate preprocessing recommendations across different data structures, cardinalities, and missing value patterns.

Most notably, when prompted with an open-ended EDA request, the agent autonomously sequenced tool calls in a logical order — structure → missingness → distributions → correlations → recommendations — without any explicit ordering instructions. This emergent sequencing behaviour suggests that tool design and natural language descriptions alone may be sufficient to guide reliable, explainable EDA decisions. This observation forms the central phenomenon under investigation in future work.

Limitations & Future Work

- The agent's follow-up reasoning can reflect dataset-specific knowledge from the LLM's training data rather than purely deriving insights from the tools. This may reduce generalisability when applied to non-standard datasets.
- DataBuddy has been successfully deployed to Hugging Face Spaces and is publicly accessible. Visualisation tools currently return plots to the chat interface via Gradio's image support, though rendering consistency across datasets with limited numeric columns requires further refinement.
- Plot generation for datasets with fewer than two numeric columns produces trivial or empty visualisations. A future iteration should include pre-checks that skip visualisation tools when the dataset structure does not support them.

Baseline Comparison

The raw ReAct implementation required manual construction of three core components: the reasoning loop, the system prompt, and the response parser.

The reasoning loop was responsible for iteratively calling the LLM, executing tools, and feeding results back into the context. The system prompt explicitly instructed the model on its role, available tools, and expected output format. The response parser

extracted tool names and arguments from the LLM's unstructured text output to enable programmatic tool execution.

LangChain abstracts all three components. The ReAct loop is handled internally by `create_react_agent`, the system prompt is managed via the hub template, and structured function calling replaces manual parsing by returning tool calls as typed objects rather than raw text.

The raw implementation offers greater transparency and control, every decision point is explicit and debuggable. LangChain reduces code complexity significantly but introduces abstraction layers that obscure the underlying reasoning process.

For production systems, LangChain is preferable for maintainability. For research purposes, the raw implementation provides clearer insight into agent behaviour and failure modes.